

RobotCar IoT

Sistema de control, monitoreo y automatización de un vehículo robótico ESP8266

Autor: Misael López Cancino

Plataforma: ESP8266 NodeMCU · Node.js · MySQL

Fecha: Mayo 2026

[Bootstrap 5](#)

[OOP](#)

[WebSocket](#)

[Chart.js](#)

[MySQL](#)

[L9110S](#)

[HC-SR04](#)

Tabla de contenido

1. [Resumen ejecutivo](#)
2. [Arquitectura general](#)
3. [Hardware utilizado](#)
4. [Stack tecnológico](#)
5. [Estructura del proyecto](#)
6. [Base de datos](#)
7. [Backend Node.js](#)
8. [Firmware ESP8266](#)
9. [Frontend \(Bootstrap + OOP\)](#)
10. [Configuración de red \(Mobile Hotspot Windows\)](#)
11. [Pasos de instalación desde cero](#)
12. [Pruebas funcionales](#)
13. [Decisiones técnicas clave](#)
14. [Glosario rápido](#)

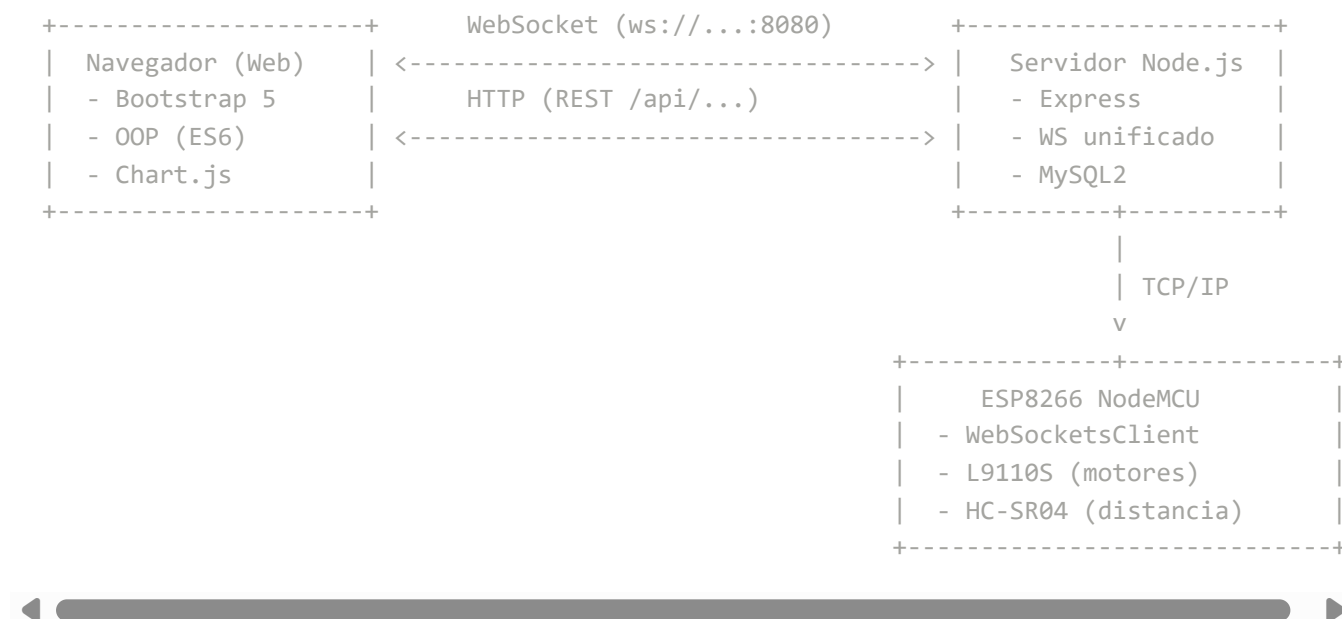
1. Resumen ejecutivo

RobotCar IoT es un sistema completo que permite **controlar, monitorear y automatizar** un vehículo robótico basado en ESP8266 desde una aplicación web. El proyecto integra:

- **Hardware:** chasis de 2 motores DC controlados por driver L9110S + sensor HC-SR04.
- **Firmware:** sketch Arduino para ESP8266 NodeMCU con cliente WebSocket.
- **Backend:** Node.js con Express + WebSockets + MySQL.
- **Frontend:** Bootstrap 5, programación orientada a objetos (ES6) y Chart.js.

El sistema cubre **CRUD de dispositivos IoT, control remoto, rutinas automáticas (demos), monitoreo en tiempo real con histórico y detección de obstáculos con alerta inmediata.**

2. Arquitectura general



Todos los componentes se comunican por la misma red WiFi (en producción: **Mobile Hotspot de Windows** llamado **RobotCar**).

3. Hardware utilizado

Componente	Modelo	Pines NodeMCU
Microcontrolador	ESP8266 NodeMCU v1.0 (ESP-12E)	—
Driver de motores	L9110S (dual H-Bridge)	D5, D2, D7, D6 (GPIO 14, 4, 13, 12)
Motores DC	2 × 3-6 V con caja reductora	conectados al L9110S
Sensor ultrasónico	HC-SR04	TRIG=D1 (GPIO 5), ECHO=D0 (GPIO 16)
Alimentación	4 × AA o batería 7.4 V	VIN + VCC del L9110S

Importante: alimentar el ESP por USB de la laptop puede causar *brownouts* cuando arrancan los motores. Se recomienda fuente externa o cargador 5 V / 1 A.

4. Stack tecnológico

Backend

- Node.js ≥ 18
- **Express 4.19.2** — servidor HTTP/REST
- **ws 8.18.0** — WebSocket unificado en el mismo puerto HTTP
- **mysql2 3.11.0** — driver con **Promise** y transacciones

Frontend

- Bootstrap 5.3.3 + Bootstrap Icons 1.11.3 (CDN)
- Chart.js 4.4.3 (CDN)
- ES6 Classes: **ApiClient** , **RealtimeBus** , **Layout** , **AdminDispositivos** , **AdminParametros** , **DeviceCard** , **ControlPanel** , **DemosManager** , **MonitorView**

Firmware

- Arduino IDE + soporte **ESP8266 Boards 3.x**
- WebSocketsClient (Markus Sattler)
- ArduinoJson 6.x

Base de datos

MySQL 8.x local (puerto 3306), base **robotcar** .

5. Estructura del proyecto

```
ESP8266/
├── Test_L9110S_Motor_Driver_Module/
│   └── Test_L9110S_Motor_Driver_Module.ino    ← firmware principal
├── WiFiScan/
│   └── WiFiScan.ino                          ← sketch diagnóstico de WiFi
└── server/
    ├── server.js                            ← arranque + WS unificado
    ├── routes.js                            ← endpoints REST
    ├── db.js                                ← pool MySQL
    ├── state.js                             ← estado en memoria
    ├── package.json
    ├── .env                                 ← credenciales locales
    ├── sql/
    │   ├── schema.sql
    │   ├── dispositivos.sql
    │   ├── demos_obstaculos.sql
    │   └── historial.sql
    └── public/
        ├── index.html, admin.html, control.html, demos.html, monitoreo.html
        ├── favicon.svg
        ├── css/styles.css
        └── js/
            ├── api.js, admin.js, control.js, demos.js, monitor.js
```

6. Base de datos

La base se llama **robotcar** y se construye con 4 scripts SQL aplicados en orden:

Script	Tablas	Contenido
schema.sql	movimientos, parametros	11 movimientos pre-cargados + parámetros del motor (Velocidad, Factor tiempo, etc.).
dispositivos.sql	dispositivos, estatus_log	CRUD de dispositivos IoT + historial de ON/OFF.
demos_obstaculos.sql	demos, demo_pasos, obstaculos	3 demos pre-cargadas (Cuadrado, Zigzag, Spin Show) + tabla de obstáculos.
historial.sql	movimientos_log, demos_log	Histórico de ejecuciones con origen y estado.

7. Backend Node.js

7.1 `server.js`

- Levanta Express en el puerto **8080**.
- Sirve `public/` como estáticos.
- Monta las rutas REST en `/api`.
- Crea servidor WebSocket en el **mismo puerto** con `ws` (`noServer:true`).
- Clasifica clientes:
 - `{"tipo":"esp"}` → guarda en `state.esp`
 - `{"tipo":"obstaculo", distancia}` → guarda en BD, broadcast, aborta demo activa
 - Otros → se agregan a `state.webClients`

7.2 `state.js`

```
module.exports = {  
  esp: null,           // conexión WebSocket del ESP  
  webClients: new Set(), // navegadores conectados  
  ultimoMovimiento: null,  
  demoActiva: null,    // { demoId, logId, abort(motivo) }  
  ultimoObstaculo: null  
};
```

7.3 Endpoints REST principales

Método	Ruta	Descripción
GET/POST/PUT/DELETE	/api/movimientos[/id]	CRUD de movimientos
POST	/api/ejecutar/:id	Ejecutar movimiento + log
GET	/api/ultimo-movimiento	Último movimiento (memoria)
GET/POST/PUT/DELETE	/api/parametros[/factor]	CRUD parámetros del motor
GET/POST/PUT/DELETE	/api/dispositivos[/id]	CRUD de dispositivos IoT
PUT	/api/dispositivos/:id/estado	ON/OFF + log + notificar ESP
GET	/api/dispositivos/:id/historial? limit=10	Histórico de cambios
GET	/api/monitoreo	Snapshot completo (todos + últimos 10)
GET/POST/PUT/DELETE	/api/demos[/id]	CRUD de demos (transaccional)
POST	/api/demos/:id/ejecutar	Ejecuta demo paso a paso (async)
POST	/api/demos/cancelar	Aborta demo activa
GET	/api/obstaculos?limit=10	Últimos obstáculos
GET	/api/obstaculos/estadisticas	Total / hoy / promedio / último
POST	/api/obstaculos	Simular obstáculo manual
GET	/api/historial/movimientos?limit=10	Últimos N movimientos
GET	/api/historial/demos?limit=10	Últimas N demos

7.4 Ejecución de demos (asíncrona y cancelable)

```

state.demoActiva = { demoId, logId, abort: motivo => { cancelado = motivo; } };

for (let i = 0; i < pasos.length; i++) {
  if (cancelado) break;
  enviarMovimientoAlEsp(p.movimiento_id);
  broadcastEvento('demo', { estado:'paso', paso:i+1, total:pasos.length });
  await new Promise(r => setTimeout(r, p.duracion_ms));
}
enviarMovimientoAlEsp(3); // Detener al final

```

Si llega un obstáculo desde el ESP, `state.demoActiva.abort('obstaculo')` interrumpe el bucle inmediatamente.

8. Firmware ESP8266

8.1 Configuración

```
const char* ssid = "RobotCar";
const char* password = "12345678";
const char* websocket_server = "192.168.137.1";
const uint16_t websocket_port = 8080;
```

8.2 Detección de obstáculos (HC-SR04)

```
float medirDistanciaCm() {
    digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);
    long dur = pulseIn(ECHO_PIN, HIGH, 30000);
    return dur ? dur * 0.0343 / 2.0 : 0;
}

// En loop(), cada 200 ms:
if (d > 0 && d < 20.0 /* cm */) {
    detenerMotores();
    if (millis() - ultimoEnvioObstaculo > 1500) {
        enviarObstaculo(d);
    }
}
```

8.3 Inicialización WiFi robusta

```
WiFi.persistent(false);
WiFi.mode(WIFI_OFF); delay(200);
WiFi.mode(WIFI_STA);
WiFi.setSleepMode(WIFI_NONE_SLEEP);
WiFi.disconnect(true); delay(200);
WiFi.begin(ssid, password);
```

Reintenta hasta 20 s; si falla imprime el `Status code` para diagnosticar.

9. Frontend (Bootstrap + OOP)

9.1 ApiClient

Encapsula `fetch` con manejo de errores y serialización JSON automática. Un método por cada endpoint REST.

9.2 RealtimeBus extends EventTarget

WebSocket auto-reconectable cada 2 s. Cuando llega `{evento:"X", ...}`, dispara un `CustomEvent('X', { detail })` que cualquier vista puede escuchar.

9.3 Layout

Genera dinámicamente el navbar (Inicio, Administración, Control, Demos, Monitoreo) y el footer con copyright.

9.4 Páginas

- **index.html** — dashboard con 4 tarjetas grandes.
- **admin.html** — 2 pestañas: dispositivos IoT (CRUD) y parámetros del motor (CRUD inline).
- **control.html** — `DeviceCard` con switch ON/OFF + botonera de movimientos.
- **demos.html** — `DemosManager` : listar, ejecutar, editar, eliminar; progreso en vivo por WS.
- **monitoreo.html** — `MonitorView` : refresco cada 2 s con KPIs, gráfico Chart.js stepped, historial por dispositivo, KPIs de obstáculos y **últimos 10 movimientos + últimas 10 demos**.

10. Configuración de red (Mobile Hotspot de Windows)

Tras múltiples problemas con hotspots de celulares (algunos Samsung bloquean clientes no-Galaxy por PMF/WPA3-transition), se optó por el **Mobile Hotspot integrado en Windows**. La laptop sirve simultáneamente como AP y como servidor.

1. Windows → **Configuración** → **Red e Internet** → **Punto de acceso móvil**.
2. Editar:
 - Nombre: `RobotCar`
 - Contraseña: `12345678`
 - **Banda: 2.4 GHz** ← obligatorio (ESP8266 no soporta 5 GHz)
3. "Compartir desde" → la WiFi del lugar.
4. **Desactivar "Ahorro de energía"**.

5. Activar el switch.
6. La IP del hotspot siempre es **192.168.137.1** (default de Windows).
7. Abrir el puerto en el Firewall (PowerShell como Administrador):

```
New-NetFirewallRule -DisplayName "RobotCar 8080" -Direction Inbound -Protocol TCP -L
```



11. Pasos de instalación desde cero

11.1 Servidor

```
# 1. Instalar Node.js 18+ y MySQL 8
# 2. Crear base de datos
mysql -u root -p
> CREATE DATABASE robotcar CHARACTER SET utf8mb4;
> exit;

# 3. Aplicar scripts SQL en orden
cd c:\Users\canci\OneDrive\Desktop\ESP8266\server\sql
mysql -u root -p robotcar < schema.sql
mysql -u root -p robotcar < dispositivos.sql
mysql -u root -p robotcar < demos_obstaculos.sql
mysql -u root -p robotcar < historial.sql

# 4. Configurar .env
# DB_HOST=127.0.0.1, DB_USER=root, DB_PASS=..., DB_NAME=robotcar, PORT=8080

# 5. Instalar dependencias y arrancar
cd ..
npm install
node server.js
```

11.2 Firmware ESP8266

1. Arduino IDE → Preferencias → URL de gestores adicionales:
http://arduino.esp8266.com/stable/package_esp8266com_index.json
2. Herramientas → Gestor de tarjetas → instalar **ESP8266**.
3. Bibliotecas → instalar **WebSockets** (Markus Sattler) y **ArduinoJson**.
4. Abrir [Test_L9110S_Motor_Driver_Module.ino](#).
5. Placa: **NodeMCU 1.0 (ESP-12E)**, puerto COM, **Erase Flash: All Flash Contents** (primera vez).
6. Subir.

12. Pruebas funcionales

#	Acción	Resultado esperado
1	Encender ESP	Serial: WiFi conectado , IP 192.168.137.x , [WS] Conectado .
2	Abrir http://192.168.137.1:8080	Dashboard carga sin errores.
3	Control → presionar "Adelante"	Carrito se mueve; aparece en "Últimos 10 movimientos".
4	Control → activar switch del carrito	Cambia a ON; se ve en Monitoreo.
5	Demos → "Cuadrado" → Ejecutar	Progreso paso a paso; al final aparece como finalizada .
6	Acercar mano <20 cm durante una demo	Carrito se detiene; alerta roja; demo abortada motivo obstaculo .
7	Admin → Parámetros → cambiar Velocidad	Valor se guarda y se aplica al próximo movimiento.

13. Decisiones técnicas clave

- **OOP en frontend:** las clases agrupan responsabilidades y permiten reusar **ApiClient** y **RealtimeBus** en todas las páginas.
- **WebSocket unificado con HTTP:** evita un segundo puerto y firewall extra.
- **Demos asíncronas con `state.demoActiva`** : cancelación inmediata desde otro endpoint o por evento de obstáculo, sin bloquear el event loop.
- **`pool.getConnection()` para transacciones de demos:** atomicidad al reemplazar pasos.
- **Mobile Hotspot de Windows:** tras descartar configuración correcta (2.4 GHz, WPA2, datos activos), se confirmó que algunos celulares bloquean clientes no-Galaxy por PMF. La laptop como AP es más confiable.
- **Cooldown de 1.5 s en obstáculos:** evita inundar el servidor cuando el obstáculo es prolongado.

14. Glosario rápido

- **PWM:** Pulse Width Modulation — control de velocidad por ancho de pulso (0–255 en Arduino).
- **L9110S:** driver dual H-Bridge para 2 motores DC.

- **HC-SR04:** sensor ultrasónico, mide distancia por tiempo de eco.
 - **WebSocket:** canal bidireccional persistente sobre TCP.
 - **Mobile Hotspot:** punto de acceso WiFi creado por software desde la laptop.
-

Documento generado para el proyecto **RobotCar IoT** — © 2026 Misael López Cancino